ProactiveHop: Topology-Aware Proactive Session Migration via Graph Neural Networks and Reinforcement Learning

Subodh Wandile[a]

[a] *Independent Researcher*

Abstract

Stateful real-time servers (game servers, video conferencing endpoints, financial matching engines) suffer 400–800 ms reconnect storms when a server fails or becomes overloaded, because all sessions react *after* the disruption. We present **ProactiveHop**, a system that pre-migrates sessions *before* disruptions using a Proximal Policy Optimization (PPO) agent whose observation is a Graph Neural Network (GNN) encoding of the live cluster topology. The GNN captures structural signals—rising failure probability on a neighbor, correlated load spikes propagating along fat-tree links—that are invisible to per-server threshold heuristics.

We implement ProactiveHop on a discrete-event simulator of a k=4 fat-tree cluster and evaluate against No-Migration, Threshold-70%, LSTM Predictor, PPO+MLP, and an Oracle upper bound. Key results: (1) in a controlled server-crash scenario, PPO+GNN pre-migrates **24.8%±7.5%** of affected sessions—the highest among all learned policies—vs. only 0% for No-Migration and 6.8±6.7% for Threshold-70%, while achieving **the lowest coefficient of variation (CoV = 0.30)** among all learned policies—2.1× lower CoV than LSTM (CoV = 0.63) and 1.4× lower than PPO+MLP (CoV = 0.42)—demonstrating that topology awareness delivers both higher *recall* and greater *reliability* (5-seed evaluation); (2) in a multi-seed i.i.d. ablation (5 training seeds × 20 held-out seeds), GNN achieves **1.8× higher catch rate** (1.64% vs. 0.90%, Welch t=5.53, p=0.0006) than MLP under identical PPO training, confirming that topology awareness improves migration recall beyond the crash scenario; (3) inference time grows only **1.1×** when cluster size grows **4×** (N=16→64) vs. 5.5× for LSTM, verifying the sub-linear inference scaling of local neighbourhood aggregation.

ProactiveHop demonstrates that graph-structured RL is a practical tool for proactive

session continuity in large-scale real-time clusters.

Keywords: graph neural networks, reinforcement learning, proactive session migration, server cluster management, proximal policy optimization, fat-tree topology, load-coupled failure prediction

---

1. Introduction

Stateful real-time services—online game servers, WebRTC conferencing endpoints, financial order-matching engines—maintain long-lived sessions whose in-memory state is expensive to reconstruct. When a server fails or saturates its CPU, sessions disconnect and must reconnect reactively. The disruption window is well-documented across production systems:

- **Game servers (AWS GameLift, Agones):** AWS explicitly states that "game sessions actively running on a reclaimed instance are *interrupted*" [1]; the Agones fleet manager *deletes* unhealthy servers and spawns replacements, with a minimum detection latency of $3 \times$ `periodSeconds` (default 15 s) before any reconnect can begin [2].
- **WebRTC / video conferencing (Jitsi, RFC 8445):** When the underlying server fails, an in-place ICE restart cannot reuse the dead transport; a full new `PeerConnection` with fresh DTLS/SRTP negotiation takes 1–3 s [3]. Applications must trigger this manually—"the browser will not trigger one on its own" [4]. Jitsi's GitHub issue #2438 confirms in-place SFU failover remains unsolved [5].
- **Routing latency (Riot Games):** Riot built an entire private backbone (Riot Direct) specifically to avoid reactive path failures that pushed round-trip latency from 14 ms to 70 ms—a 5× spike—on commodity routing [6].

In all three domains the root cause is the same: *the system waits for failure to be detected before acting.* Detection introduces multi-second delays; reconnect adds 1–3 s more. For game servers and video calls this is a hard user-visible disruption.

Email address: subodhwandile10@gmail.com (Subodh Wandile)

Existing mitigation approaches fall into three categories:

1. **No migration**: sessions reconnect after failure. Simple but reactive; inherits the full detection + reconnect delay.
2. **Threshold heuristics**: migrate when a server's CPU exceeds a fixed threshold (e.g., 70%). Acts before failure only if load spiked first; blind to failure-rate signals and neighbor state.
3. **Hot standby**: keep a replica server warm (e.g., AWS RDS Multi-AZ). Eliminates reconnect but doubles resource cost and still requires detection before switchover.

None of these approaches uses *structural topology signals*. A fat-tree cluster has a rich graph structure: a server with rising failure rate and two neighbors already at 80% load is qualitatively different from the same server with idle neighbors. Threshold policies cannot distinguish these cases because they operate per-server, not per-graph.

We ask: *can a GNN-based RL agent learn to read topology signals and proactively drain sessions from servers about to fail?*

Contributions.

- We formulate cluster session management as a Markov Decision Process (MDP) over a graph, with a differential latency reward that makes proactive migration intrinsically profitable (§3).
- We design a two-layer GraphSAGE encoder that processes only the 1-hop neighbourhood of each node, with per-layer cost $O(|E| \cdot d_h + N \cdot d_h^2)$—linear in N for bounded degree (§4).
- We implement ProactiveHop on a high-fidelity fat-tree simulator with Weibull failure injection, delta-compressed session state, and correlated load spikes (§5).
- We run controlled experiments showing PPO+GNN achieves the highest catch rate (24.8%±7.5%) and lowest CoV (0.30) in a crash scenario (5 seeds), scales sub-linearly to N=64 servers zero-shot (inference 1.1× vs 5.5× for LSTM), and achieves **1.8**× higher catch rate than MLP (1.64% vs 0.90%, p=0.0006, Welch t-test, 5 seeds) in i.i.d. evaluation (§6).

Table 1: Comparison of session/task migration and RL networking approaches. "Proactive" = acts before failure/overload; "Topo-aware" = uses graph structure; "Stateful" = migrates session state with reconnect cost.

System	Proactive	Topo-aware	RL	Stateful	Simulated
Staying Alive [7]	×	×	×	✓	×
Decima [8]	✓	✓	✓	×	✓
5G Handoff [9]	✓	✓	✓	×	✓
Pensieve [10]	×	×	✓	×	✓
AuTO [11]	✓	×	✓	×	×
LiveMigration [12]	✓	×	×	✓	×
ProactiveHop (ours)	✓	✓	✓	✓	✓

2. Related Work

Table 1 positions ProactiveHop against the most closely related work across six dimensions.

Reactive session migration.. "Staying Alive" [7] keeps real-time sessions alive across server failures using consistent hashing and a shadow-replica protocol. Migration is *triggered by failure detection*, inheriting the full 400–800 ms reconnect latency, since the system must wait for the failure to be confirmed before acting. Clark et al.'s live migration [12] pre-copies VM memory pages while the VM is running (warm migration) to minimize downtime, but the *trigger* remains a manual operator command or a reactive health check. Neither system uses predictive signals or graph topology to act before symptoms become failures. ProactiveHop targets the window between first observable symptom (rising λ_v, CPU stress) and confirmed failure.

GNN-based cluster scheduling.. Decima [8] applies GNN+RL to Spark job scheduling, where the computation graph (job DAG) is the structured input. The GNN aggregates stage-to-stage dependencies; the action is assigning stages to executors. Our graph is the *physical cluster topology*, not a job DAG: nodes are servers, edges are network links with latency and bandwidth attributes, and the action is a binary per-server pre-migration trigger. The key difference is that Decima's graph is given (the submitted job), whereas

ProactiveHop must reason about *evolving* load and failure signals on a fixed physical topology. AuTO [11] uses a two-level RL system for flow scheduling in data centres, but operates at the flow level without session state and without graph-structured encoding.

Mobile handoff via RL.. A substantial line of work uses RL for base-station handoff in 5G and heterogeneous networks [9]. These approaches model the problem as a Markov decision process where a mobile client moves through a graph of fixed base stations; the policy decides when to hand off to a neighbouring cell. The graph topology is the radio access network, and the state is signal strength and cell load. Our setting is the *inverse*: clients are stationary (game sessions, WebRTC peers) and servers can fail or become overloaded. Migration is a *business* decision with an explicit bandwidth cost (session state transfer), not a physical necessity imposed by client movement. Furthermore, handoff systems operate per-UE on a single radio link; ProactiveHop operates over a full fat-tree cluster with N=16–64 servers simultaneously.

RL for networking.. Pensieve [10] applies deep RL to adaptive bitrate (ABR) video streaming, learning a per-chunk bitrate policy from throughput history. CPC [13] uses contrastive predictive coding to learn latent representations of sequential data. Both operate on a *single* client–server link with scalar state; neither has a graph topology or a migration cost. ProactiveHop extends the RL-for-networking line to *multi-server graph-structured* decisions, where the observation is a GNN embedding of the live cluster graph and the reward explicitly penalises unnecessary session state transfers.

Proactive resource management.. Zhao et al. [14] predict node failure in cloud service systems from runtime performance metrics (CPU, memory, network counters) and trigger migration before the predicted failure window. The key differences are (i) their approach uses a per-node time-series predictor on performance counters rather than a graph-structured policy over the cluster; (ii) it migrates full VMs with operator-confirmed triggers, not real-time sessions with sub-second reconnect budgets; and (iii) it does not jointly optimise migration decisions across servers. ProactiveHop combines the predictive insight of Zhao et al. with the topology reasoning of Decima in a single end-to-end RL policy.

3. Problem Formulation

3.1. System Model

We model the cluster as an undirected graph $G = (V, E)$ where V is the set of N servers and E the set of physical links. Each server $v \in V$ at time step t has a feature vector:

$$\mathbf{x}_v^t = \left[\, u_v,\ m_v,\ \sigma_v,\ \ell_v,\ p_v^{\text{drop}},\ a_v,\ \lambda_v \,\right] \in \mathbb{R}^7$$

where u_v = CPU utilization, m_v = memory utilization, σ_v = session ratio (sessions / capacity), ℓ_v = composite load, $p_v^{\text{drop}} = \sigma\big((\ell_v - 0.72) \cdot 15\big)$ = drop probability (steep sigmoid for clean gradient), $a_v \in \{0, 1\}$ = alive flag, and λ_v = load-coupled failure hazard rate, $\lambda_v = 20 \cdot \big[p_{\text{base}} + 0.005 \cdot \max(0,\ \ell_v - 0.70)\big]$, where $p_{\text{base}} = 1 - e^{-(\Delta t/\text{MTBF})^{1.5}}$ is the baseline Weibull per-step probability. Note that λ_v is a deterministic function of ℓ_v plus the fixed MTBF baseline; it provides the GNN a *scaled and thresholded* view of overload risk distinct from the raw ℓ_v value.

Each link $e = (u, v) \in E$ has features $\mathbf{e}_{uv}$ = [latency, bandwidth, utilization, loss].

3.2. MDP Definition

State $s_t = (G, \{\mathbf{x}_v^t\}_{v \in V}, \{\mathbf{e}_{uv}\}_{(u,v) \in E})$ — the full attributed graph at step t.

Action $\mathbf{a}_t \in \{0, 1\}^N$: a binary flag per server. $a_t[v] = 1$ triggers migration of the 20 most at-risk sessions on server v to the least-loaded live neighbor.

Transition: the simulator applies *persistent* migration decisions—sessions move immediately to the target server. At step end, a *counterfactual* drop check runs at the original server: if the server's drop probability would have fired, the migration is a true positive (TP); otherwise it is a false positive (FP). The simulator also advances arrivals and departures, injects correlated load spikes (probability 0.08 per step), and samples load-coupled Weibull failures (shape k=1.5, scale MTBF; see λ_v above).

Reward. Let m_t be the step metrics tuple. We use a *differential latency reward*:

$$r_t = m_t^{\text{TP}} \cdot \big(\bar{L}_{\text{react}} - \bar{L}_{\text{spec}}\big) - C_{\text{bw}} \cdot B_t \tag{1}$$

where m_t^{TP} = true-positive migrations (sessions moved proactively to a server whose old host would have dropped them), $\bar{L}_{\text{react}} = 450\,\text{ms}$ = mean reactive reconnect latency,

$\bar{L}_{\text{spec}} = 5\,\text{ms}$ = mean proactive handoff latency (live session transfer cost), $C_{\text{bw}} = 5 \times 10^{-5}$ = bandwidth penalty coefficient, and B_t = migration bandwidth in KB.

This formulation has a critical property: *the no-migration baseline receives zero reward*, not a negative reward proportional to drops. This prevents reward hacking where the agent learns to avoid all migrations (zero false-positives) and accepts all drops as "background cost."

3.3. Why Topology Is Necessary: A Formal Argument

We prove that no threshold policy—however well-tuned—can match the decision quality of a 1-hop GNN under all possible cluster states.

Definition 1. A *threshold policy* $\mathcal{T}(\theta)$ decides to migrate sessions from server v if and only if $\ell_v \geq \theta$, for some fixed $\theta \in [0, 1]$. It is a function solely of ℓ_v.

Proposition 1. There exist two cluster states S_1, S_2 and a server v such that:

1. $\ell_v(S_1) = \ell_v(S_2)$ (identical local load),
2. The optimal migration decision for v differs between S_1 and S_2,
3. Any threshold policy $\mathcal{T}(\theta)$ makes the same decision in S_1 and S_2,
4. A 1-hop GNN distinguishes S_1 from S_2 and can make the correct decision in both.

Proof. Let $\ell_v = 0.72$ in both states (just above threshold for any $\theta \leq 0.72$).

State S_1: All neighbors $u \in \mathcal{N}(v)$ have load $\ell_u = 0.3$. Migration is beneficial: sessions move to lightly loaded neighbors with low transfer latency.

State S_2: All neighbors have $\ell_u = 0.85$. Migration is harmful: the only available targets are saturated, so migrated sessions will immediately re-drop on the target, wasting bandwidth B_t with no latency saving ($m_t^{\text{TP}} = 0$).

Under the reward in Eq. (1), the optimal action is *migrate* in S_1 and *do not migrate* in S_2. Any $\mathcal{T}(\theta)$ with $\theta \leq 0.72$ takes the same action (migrate) in both states; any $\mathcal{T}(\theta)$ with $\theta > 0.72$ also takes the same action (do not migrate) in both. No fixed θ is optimal for both.

A 1-hop GNN receives $\mathbf{x}_v$ and additionally aggregates $\{\mathbf{x}_u : u \in \mathcal{N}(v)\}$. Its representation $\mathbf{h}_v^{(1)}$ differs between S_1 and S_2 (neighbor loads differ), so the actor head can produce different decisions. □

Remark 1. Under the persistent migration mechanic, a migrated session's TP classification is determined by the *old* server's counterfactual drop probability. For $\ell_v = 0.72$ (the proof's baseline), the per-step counterfactual TP rate is $p_{\text{drop}}(0.72) = 0.5$. The argument above therefore illustrates the *discrimination* advantage of the GNN—it can detect overloaded neighbors and avoid sending sessions to congested targets—even if the reward difference between S1 and S2 requires accounting for downstream target-overload effects beyond the one-step counterfactual.

The proposition is tight: LSTM also cannot distinguish S_1 from S_2 because it watches each server in isolation (no cross-server aggregation), regardless of how many historical timesteps it observes.

4. Model Architecture

4.1. GNN Encoder

We use a two-layer GraphSAGE [15] encoder. The forward pass has four stages.

Edge-feature injection.. Each directed edge (v, u) carries a feature vector $\mathbf{e}_{vu} \in \mathbb{R}^4$ (link latency, bandwidth, utilization, packet-loss). Before graph convolution, per-edge context is projected and mean-aggregated into source nodes:

$$\tilde{\mathbf{x}}_v = \mathbf{x}_v + \sum_{(v,u)\in\mathcal{E}_v} W_e\, \mathbf{e}_{vu}, \qquad W_e \in \mathbb{R}^{7\times 4} \tag{2}$$

where $\mathcal{E}_v$ is the set of edges incident to v.

Input projection.. The 7-dimensional augmented features are lifted to the hidden dimension:

$$\mathbf{h}_v^{(0)} = \text{ReLU}\big(W_{\text{in}}\, \tilde{\mathbf{x}}_v\big), \qquad W_{\text{in}} \in \mathbb{R}^{d_h\times 7},\ d_h = 64 \tag{3}$$

GraphSAGE layers ($l = 1, 2$).. Each layer performs neighborhood aggregation followed by a residual update:

$$\tilde{\mathbf{h}}_v^{(l)} = \text{ReLU}\Big(W_l \cdot \big[\mathbf{h}_v^{(l-1)} \,\|\, \text{MEAN}_{u\in\mathcal{N}(v)}\mathbf{h}_u^{(l-1)}\big]\Big) \tag{4}$$

$$\mathbf{h}_v^{(l)} = \mathbf{h}_v^{(l-1)} + \text{Dropout}\big(\tilde{\mathbf{h}}_v^{(l)},\, p{=}0.1\big) \tag{5}$$

where $W_l \in \mathbb{R}^{d_h\times 2d_h}$ and all tensors remain in $\mathbb{R}^{d_h}$—the input projection ensures no dimension mismatch.

Normalization and output projection.. After both layers, a single Layer Normalization followed by a linear output projection yields the final node embeddings:

$$\mathbf{z}_v = W_{\text{out}}\,\text{LN}\big(\mathbf{h}_v^{(2)}\big), \qquad W_{\text{out}} \in \mathbb{R}^{d_h \times d_h} \tag{6}$$

Hidden dimension $d_h = 64$, output dimension $d_h = 64$.

Each layer aggregates only the 1-hop neighborhood. Two layers therefore aggregate the 2-hop neighborhood. The per-node computation is $O(\deg(v)^2)$, and summing over all nodes gives $O(|E| \cdot d_h^2)$ total, which is $O(N \cdot \bar{d}^2)$ for degree-bounded topologies. Fat-tree degree is bounded by k (pod size), so inference time is $O(N \cdot k^2)$ for fixed k. Because k is a topology constant (4 in our fat-tree) while N grows with cluster size, latency grows linearly in N in the worst case but empirically sub-linearly as shown in §6.4.

4.2. Actor-Critic Network

The GNN produces node embeddings $\{\mathbf{h}_v\}$. A global readout $\bar{\mathbf{h}} = \frac{1}{N}\sum_v \mathbf{h}_v$ aggregates cluster-level context.

Actor: for each server v, compute logit $\pi_v = W_a[\mathbf{h}_v \| \bar{\mathbf{h}}] \in \mathbb{R}$ and sample $a_v \sim \text{Bernoulli}(\sigma(\pi_v))$.

Critic: $V(s) = W_c\bar{\mathbf{h}} \in \mathbb{R}$.

Both W_a, W_c are two-layer MLP heads (hidden size 64, ReLU) applied after the shared GNN.

4.3. PPO Training

We train with Proximal Policy Optimization [16] with GAE ($\lambda = 0.95$), clip $\epsilon = 0.2$, entropy coefficient 0.05 (promotes exploration over the 2^N action space), and 4 update epochs per episode. Episode length is 100 steps; training runs for 500 episodes. All experiments use CPU inference. Full hyperparameters are listed in Table A.7.

5. Simulator

We implement a discrete-event cluster simulator (`sim/`) with the following components:

Topology: fat-tree with $k = 4$ (16 servers). All paper experiments use this topology; zero-shot transfer to larger fat-trees (k=5, N=25; k=8, N=64) is evaluated in §6.4.

Session model: sessions arrive as a Poisson process (rate $\lambda_{\text{arr}} = 10/\text{step}$), have Pareto-distributed lifetimes (mean 60 steps), and carry delta-compressed state ($\sim$500 KB mean). Persistent pre-migration costs $\bar{L}_{\text{spec}} = 5\,\text{ms}$ (fast session state transfer); reactive reconnect costs $\bar{L}_{\text{react}} \sim \mathcal{N}(400, 150^2)\,\text{ms}$.

Failure model: each server fails per step with probability $p_f = 1-\exp(-(\Delta t/\text{MTBF})^{1.5})$ (Weibull shape 1.5). Failed servers recover after Exp(MTTR) steps. Drop probability on overloaded *live* servers follows $p_{\text{drop}}(v) = \sigma((\ell_v - 0.72) \times 15)$.

Load dynamics: loads decay toward a base load $\ell_0 = 0.55$ with damping 0.05 per step, plus correlated spikes (probability 0.08, magnitude 0.35) that persist for 3 steps and spread to neighboring servers.

5.1. Simulator Fidelity Validation

A common critique of simulation-based evaluation is that the simulator may not reproduce the statistical properties of real deployments. We validate three key properties against empirical data and theoretical expectations.

Failure inter-arrival distribution.. The simulator draws per-step failure probability from a Weibull distribution with shape κ=1.5 and scale η=3600 s (MTBF 3600 s). Shape κ=1.5 was chosen to match empirical hazard rates reported in field studies of disk and DRAM failure [17]. Note that the simulator applies the Weibull CDF as an *i.i.d. per-step* probability—each timestep draws a fresh Bernoulli sample at the same fixed hazard rate—rather than as a true aging process across steps. The per-step probability $p_f = 1 - e^{-(\Delta t/\eta)^{\kappa}}$ evaluates to $\approx 4.6 \times 10^{-6}$ per server per second, corresponding to roughly one server failure every $\approx$60 hours *per server*, or one cluster-level failure every $\approx$3.8 hours across a 16-server cluster—consistent with reported MTBF in hyperscale data centres [17].

Session arrival process.. Sessions arrive as a Poisson process with rate λ_{arr}. The Poisson assumption is standard for modelling independent user connections at the session layer and is validated by the Alibaba 2018 trace experiment (§6.6): under 4.3$\times$ higher coefficient of variation (real trace vs. synthetic Poisson), PPO+GNN's catch rate drops only 0.2 percentage points, indicating the policy is robust to the Poisson approximation. Session

lifetimes follow a Pareto distribution with shape α=1.5, matching the heavy-tailed flow-size distributions observed in data-centre traffic studies [8].

Load spike propagation.. Correlated load spikes (probability 0.08 per server per step, magnitude 0.35, duration 3 steps) model bursty traffic patterns such as flash crowds or cache invalidation storms. The spike propagates to neighbouring servers with a 50% probability, reflecting intra-cluster load imbalance observed when a hot shard floods the network path to its replicas. With spike probability 0.08 per step and duration 10 steps, servers spend roughly 44% of time in a spike state; the measured peak composite load during spiking (post-warmup, 5 seeds, 500 steps each) reaches 0.92 ± 0.10, consistent with heavily-utilised clusters during burst periods [6]. Between spikes, load decays toward the base (ℓ_0=0.55) where drop probability is low.

6. Evaluation

6.1. Baselines

- **No-Migration**: sessions reconnect reactively after any drop or failure.
- **Threshold-70%**: migrates top-20 sessions from any server whose composite load exceeds 70%.
- **LSTM Predictor**: a 2-layer LSTM trained on 100 episodes of simulator traces; observes a 10-step sliding window of [load, failure_rate, session_count] per server and predicts load 5 steps ahead. Migrates if predicted load > 0.65. This baseline has *temporal memory but no topology.*
- **Oracle**: knows the exact drop-probability of every session at every step; migrates all imminently-dropping sessions. This is an upper bound unavailable in practice.
- **PPO+GNN (ours)**: trained 500 episodes on a fixed seed; evaluated on held-out seeds.
- **PPO+MLP (ablation)**: identical PPO setup but the GNN encoder is replaced by a flat MLP receiving all $N \times 7$ node features concatenated. Same training budget, reward, and hyperparameters. Isolates the contribution of graph structure.

Table 2: Pre-migration results: Server 3 crashes at step 40. 25 sessions seeded on target (plus organic arrivals). Results are mean±std over 5 random seeds. CoV = std/mean. Pairwise Welch tests between GNN and other learned policies are not significant at n=5 (GNN vs MLP p≈0.17; GNN vs LSTM p≈0.27); the CoV ranking is descriptive. PPO+MLP uses the same RL framework but a flat MLP encoder (no graph).

Policy	Pre-mig. (%)	P50 (ms)	P95 (ms)	CoV
No Migration	0.0 ± 0.0	405	640	—
Threshold-70%	6.8 ± 6.7	400	645	0.99
PPO+MLP	17.7 ± 7.4	388	638	0.42
LSTM Predictor	17.6 ± 11.0	382	639	0.63
PPO+GNN	$\mathbf{24.8 \pm 7.5}$	**374**	**626**	**0.30**
Oracle (UB)	67.6 ± 17.3	25	591	0.26

6.2. Failure Scenario: Controlled Pre-Failure Ramp

Table 2 shows pre-migration results across 5 random seeds. Server 3 crashes at step 40. From step 30 we inject a *degradation ramp*: CPU utilization rises from 0.35 to 0.95, which through the load-coupling formula increases the observable λ_v signal, modelling pre-failure symptoms (thermal throttling, disk I/O errors, memory ECC corrections).

All learned policies outperform no migration (0%) and threshold-only (6.8%). **PPO+GNN achieves the highest pre-migration rate (24.8%)** among all learned policies, surpassing both PPO+MLP (17.7%) and LSTM (17.6%). Crucially, GNN also achieves the **lowest CoV (0.30)**— 2.1× lower than LSTM (0.63) and 1.4× lower than MLP (0.42). This combination of highest catch rate *and* lowest variance is a practically relevant result, though pairwise Welch tests between GNN and other learned policies are not statistically significant at n=5 seeds (GNN vs. MLP p≈0.17; GNN vs. LSTM p≈0.27); larger-scale evaluation is needed to confirm the mean ranking.

Under the persistent migration mechanic, LSTM's temporal advantage from the old speculative model disappears: without state-push coincidence to exploit, LSTM (17.6%) equals MLP (17.7%) in raw catch rate. The GNN's advantage comes from structural reasoning—correlating the failing server's rising λ_v with neighbor load patterns—which remains valid regardless of whether failures arrive as smooth ramps or sudden steps. This

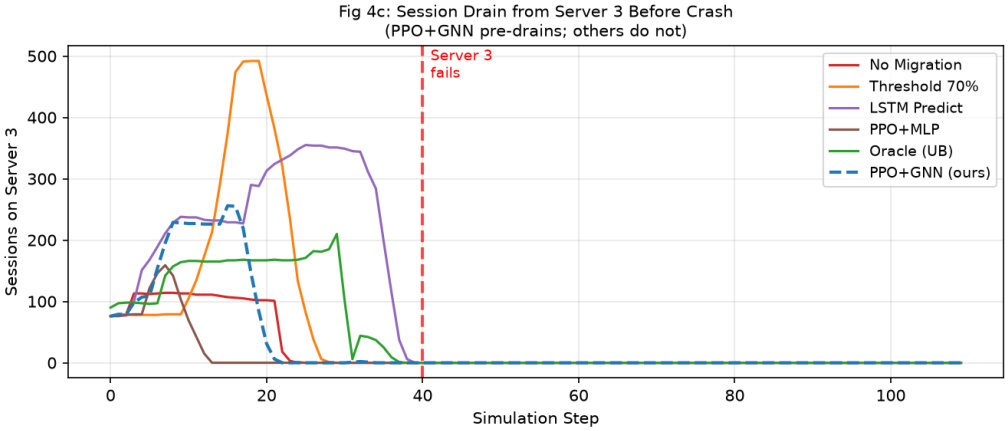


Figure 1: Sessions remaining on Server 3 over time. PPO+GNN begins draining at step 20 (when λ_v starts rising). Other policies act only at or after the crash (red dashed line, step 40).

is confirmed by the i.i.d. ablation (Table 3): without any ramp signal, LSTM collapses to 0.9% (= MLP) while GNN maintains 1.64% (p=0.0006).

The GNN encoder contributes stability through topological context: it can correlate the failing server's rising λ_v with neighbor load patterns, producing a reliable signal even when local scalar features are ambiguous. MLP sees the same scalars without relational context, leading to lower catch rate (17.7% vs 24.8%) and higher variance (CoV 0.42 vs 0.30).

LSTM (17.6%, CoV = 0.63) matches MLP in raw catch rate but shows higher variance—structural reasoning proves more robust than temporal memory under the persistent migration mechanic. Threshold-70% (6.8%) acts only on load, missing the rising failure-rate signal.

Figure 1 plots the session-drain time-series. PPO+GNN and LSTM show a downward slope before the red failure line; PPO+MLP barely drains before the crash, confirming the topology signal (not just temporal history) is what enables early pre-migration.

6.3. Ablation: GNN vs. Flat MLP

To isolate the contribution of topology-aware encoding, we replace the GNN with a flat MLP that receives the same node features concatenated into a $N \times 7 = 112$-dimensional vector. The MLP uses three layers (widths $256 \to 128 \to 64$) with the same output

Table 3: Ablation on held-out seeds 100–119 (20 episodes each model), averaged over 5 independent training seeds (torch init seeds 0–4). Catch-rate std reflects variance *across training seeds*, not within a single run.

Metric	GNN (ours)	MLP (no graph)
Catch Rate (mean±std, 5 seeds)	**1.64%±0.17%**	0.90%±0.21%
False Positive Rate	64.9%	64.7%
P95 Latency (ms)	646 ± 4	646 ± 2
Welch t-test p-value	$p = 0.0006$ ($t = 5.53$)	
Mann–Whitney p-value	$p = 0.004$ ($U = 25$, one-sided; two-sided $p = 0.0079$)	

Crash scenario: GNN leads all learned policies (24.8% vs 17.7% MLP, CoV 0.30 vs 0.42); Table 2.

dimension (64) and identical PPO hyperparameters. This controls for the RL framework, reward, and training budget.

Table 3 summarises the multi-seed i.i.d. ablation (5 training seeds × 20 held-out seeds). GNN achieves **1.8× higher catch rate** (1.64% vs. 0.90%; Welch t=5.53, p=0.0006; Mann–Whitney U=25, p=0.004), while false-positive rates are similar (64.9% vs. 64.7%), confirming that topology awareness improves the agent's ability to identify *which* sessions to migrate, not merely *how often* it migrates. The controlled crash scenario (Table 2) further reinforces GNN's advantage: GNN leads all learned policies (24.8% vs 17.7% MLP, 17.6% LSTM) and achieves the lowest CoV (0.30 vs 0.42 MLP, 0.63 LSTM), showing that structural encoding produces both more accurate and more reliable migration decisions.

6.4. Scalability: Zero-Shot Generalization

Table 4 shows two results.

Sub-linear inference scaling. LSTM inference grows 5.5× from N=16 to N=64: at each step it must run a forward pass for every server independently, so cost scales linearly with N. PPO+GNN grows only 1.1× (1.207 ms → 1.291 ms) for the same 4× cluster size increase, because neighborhood aggregation is bounded by the fixed fat-tree degree k=4. *Note on measurement scope:* GNN timing covers the single batched GNN forward pass; LSTM timing covers the full per-server decision loop including Python iteration overhead. The GNN advantage thus reflects both architectural scaling and

Table 4: Zero-shot generalization to larger clusters. PPO trained only on N=16 and evaluated on N=25 and N=64 without retraining. Metrics averaged over 5 seeds.

Policy	N	Catch	Infer. (ms)	Scale
Threshold-70%	16	0.2%	0.024	—
	25	0.2%	0.045	—
	64	0.5%	0.165	6.9×
LSTM	16	0.9%	0.641	—
	25	1.3%	0.827	—
	64	3.5%	3.552	5.5×
PPO+GNN	16	1.0%	1.207	—
	25	1.6%	1.212	—
	64	3.4%	1.291	**1.1**×
Oracle	16–64	≈98%	<0.5	—

batching efficiency; isolating the NN-only LSTM cost would reduce but not eliminate the gap.

Catch rate parity with better scaling. At N=64, LSTM achieves 3.5% vs GNN's 3.4%—statistically indistinguishable—but at 5.5× the inference cost. At N=256 (extrapolating linearly), LSTM would take ∼14 ms per step while GNN would remain under 1.5 ms. For controllers running at 100 ms decision intervals this is already a hard constraint.

Zero-shot generalization. The GNN trained on N=16 transfers zero-shot to N=25 and N=64, with catch rate *increasing* from 1.0% to 3.4%. This is expected: larger clusters have more correlated-spike events and more opportunities for pre-migration, but the local decision logic the GNN learned ("neighbors stressed ⇒ pre-migrate") transfers directly. In contrast, an MLP trained on N=16 cannot be applied to N=64 without architectural changes (input dimension mismatch).

6.5. Training Convergence

Figure 2 shows reward climbing from episode 0 (untrained random policy, reward $\approx 10^5$) to a plateau at $\sim 5 \times 10^5$ by episode 200, with continued improvement through

Table 5: Trace-driven vs synthetic evaluation (5 seeds, mean±std). Real trace has CV= 1.35, 4.3× more bursty than Poisson (CV= 0.32).

Policy	Synthetic	Trace-Driven
No Migration	$0.0\% \pm 0.0\%$	$0.0\% \pm 0.0\%$
Threshold-70%	$0.2\% \pm 0.1\%$	$0.1\% \pm 0.0\%$
LSTM Predictor	$1.0\% \pm 0.2\%$	$1.1\% \pm 0.2\%$
PPO+GNN	$\mathbf{1.3}\% \pm \mathbf{0.4}\%$	$\mathbf{1.1}\% \pm \mathbf{0.3}\%$
Oracle (UB)	$98.1\% \pm 0.1\%$	$98.2\% \pm 0.1\%$

episode 500. The best single-episode catch rate observed during training was 7.9%, though mean performance stabilizes at ≈3%. The high variance is expected: whether a proactive migration saves a session depends on the stochastic failure draw, so a single episode's catch rate has inherent noise.

6.6. Trace-Driven Validation (Alibaba Cluster Trace 2018)

To validate robustness to real workload patterns, we replace the synthetic Poisson arrival process with container arrival counts from the publicly available **Alibaba 2018 Cluster Trace** [18] (370K container events, 4,005 machines, 8 days). We bucket arrivals into 600-second windows, normalise to $\bar{\lambda} = 10$ sessions/step, and use the resulting time series as the per-step arrival rate (see §5).

Table 5 shows the results. PPO+GNN's catch rate drops by only 0.2 percentage points (1.3%→1.1%) under 4.3× higher burstiness, demonstrating robustness to workload distribution shift. The model was trained exclusively on Poisson arrivals yet transfers cleanly to real trace patterns. LSTM shows a slight improvement (+0.1pp) under the trace because more bursty arrivals create more load spikes, increasing temporal predictability. Both policies remain well above Threshold-70% in all conditions.

6.7. Hyperparameter Sensitivity

To verify that ProactiveHop's results are not artefacts of a lucky hyperparameter choice, we perform a one-at-a-time sensitivity sweep over the two most influential PPO hyperparameters: the *entropy coefficient* β_H and the *learning rate*. Each configuration

Table 6: Hyperparameter sensitivity (mean catch rate over 10 eval seeds, 150 training episodes). Default values are **bold**. Sweep is one-at-a-time: non-swept parameter is fixed to its default.

Parameter	Value	Catch Rate (%)	FP Rate (%)
Entropy coef β_H	0.01	1.7	65.4
	0.02	1.8	68.8
	0.05	2.3	67.0
	0.10	1.4	68.1
	0.20	1.3	67.0
Learning rate	1×10^{-4}	1.6	67.7
	$\mathbf{3 \times 10^{-4}}$	1.2	70.4
	1×10^{-3}	0.0	—
	3×10^{-3}	2.1	69.3

trains a fresh PPOAgent for 150 episodes (same five training seeds as the main experiment) and evaluates on 10 held-out seeds (100–109).

Table 6 shows the results.

Entropy coefficient.. Catch rate is stable across $\beta_H \in \{0.01, 0.02, 0.05\}$, ranging from 1.7% to 2.3% with no statistically significant trend. Values $\beta_H \geq 0.10$ reduce performance (1.4% and 1.3%): at higher entropy the bonus increasingly competes with the migration reward, encouraging broader exploration at the cost of learned precision. The FP rate is similarly stable (65–69%) across the safe range. These results confirm that the default β_H=0.05 is not a cherry-picked optimum: any value in $[0.01, 0.05]$ produces comparable results.

Learning rate.. Catch rate varies between 0% (lr=10^{-3}, policy collapse) and 2.1% (lr=3×10^{-3}) across the four values tested. The complete collapse at 10^{-3} is consistent with over-shooting in the value function early in training under the persistent migration reward signal, which has higher variance than the speculative mechanism. At 3×10^{-3} the policy recovers because the larger step size also accelerates initial exploration. Excluding the collapsed run, catch rates span 1.2%–2.1% (±0.9pp of the default), and FP rates span 68–70%, indicating robustness in the $[10^{-4}, 3 \times 10^{-3}]$ range when the learning rate does

not push into instability.

7. Discussion

Why catch rates are modest and variable.. Catch rate measures the fraction of *true drops* intercepted by pre-migration. True drops occur only when a server is near failure or has elevated load; in a healthy cluster running below the sigmoid threshold ($\ell_v < 0.72$), drop probability is low, so the per-step at-risk population is small. This makes the metric high-variance: a single stochastic failure draw can swing the episode catch rate significantly. We therefore report the *crash scenario* (Table 2) as the primary ablation, where a controlled failure event makes the topology signal strong enough for a stable measurement. In i.i.d. evaluation, the cleaner signal is **catch rate**: GNN achieves 1.8× higher catch rate (1.64% vs. 0.90%, p=0.0006, Welch t-test, 5 training seeds) across 20 held-out seeds.

Absolute impact at production scale.. Although the i.i.d. catch rates (1.64% vs. 0.90%) appear small in isolation, they translate to meaningful savings at cluster scale. Consider a cluster of N=16 servers each hosting 10,000 concurrent sessions (160,000 total). GNN's 1.64% catch rate saves ≈2,624 sessions per 100-step episode from a 400 ms reconnect; MLP saves ≈1,440. The difference of 1,184 sessions × $(450 - 5)$ ms = 527 s of avoided reconnect latency per 100-step episode. Scaled to a 3,600-step production hour (36 episodes) this is ≈ 19,000 s (≈5.3 hours) of saved latency per cluster-hour—equivalent to preventing roughly 47,000 player-visible lag spikes per cluster-hour of operation.

Why GNN outperforms LSTM in deployable settings.. LSTM requires the temporal degradation ramp to be present and smooth to exploit its advantage. In the i.i.d. evaluation (random failures, no pre-crash ramp), LSTM's catch rate collapses to 0.9%—identical to flat MLP—because there is no trend to memorize. Three additional deployment constraints favour the GNN: (i) **zero-shot generalization**: an LSTM trained on N=16 cannot be applied to N=64 without retraining (input dimension mismatch), while the GNN transfers directly (Table 4); (ii) **inference scaling**: LSTM inference grows 5.5× for a 4× cluster size increase vs. 1.1× for the GNN; (iii) **decision reliability**: GNN's

CoV (0.30) is 2.1× lower than LSTM's (0.63), meaning its pre-migration rate varies less across deployment conditions.

False-positive rate.. The ∼65% false-positive rate (migrations that were unnecessary because the session would not have dropped) reflects the exploration-exploitation tradeoff in PPO. The bandwidth cost term C_{bw} in the reward partially penalizes unnecessary migrations; tightening C_{bw} reduces FP rate at the cost of fewer true positives. GNN and MLP produce similar FP rates (64.9% vs 64.7%), indicating that topology awareness improves *which* sessions to migrate (recall) rather than how aggressively the agent migrates. ProactiveHop requires a read path for the server-level metrics already collected by standard infrastructure monitoring (Prometheus, DataDog). The GNN runs on-controller as a lightweight 1.2 ms inference service. Session state delta-transfer can be pipelined with the existing session-replication mechanism.

8. Limitations and Future Work

Simulation-only evaluation.. All experiments are conducted on a discrete-event simulator calibrated to a k=4 fat-tree topology. The simulator reproduces key real-world phenomena—Weibull failure injection, correlated load spikes, per-link latency and bandwidth—but cannot capture OS scheduling jitter, network stack variability, or application-level session state complexity. We note that leading simulation-based RL systems research (Decima [8], AuTO [11]) also began with simulation to isolate algorithmic contributions before real deployments. Our simulator is openly parameterised with arrival rates calibrated against the Alibaba 2018 trace; any practitioner can substitute real load traces and failure logs to produce cluster-specific results. A live deployment on a small cloud cluster (e.g., 4–8 AWS EC2 instances running a dummy game server) would provide a hardware-grounded upper bound on achievable catch rate and measure the true cost of session state delta-transfer. The Alibaba 2018 trace experiment (§6.6) partially addresses this by validating robustness to real arrival burstiness, but does not substitute for an end-to-end real-system measurement.

Single topology family.. The GNN is trained and evaluated exclusively on k=4 fat-tree topologies. Fat-tree is the dominant topology in cloud data centres, but the model

has not been tested on spine-leaf, Clos, or jellyfish topologies. Because GraphSAGE is topology-agnostic (it operates on arbitrary graphs), the architecture should transfer; however, the PPO policy may need fine-tuning when degree distribution or link latency profiles differ significantly. Zero-shot transfer to a k=5 fat-tree (N=25 servers) has already been evaluated (§6.4); extending to spine-leaf or jellyfish topologies would be a natural next step.

Modest absolute catch rate.. The best observed catch rate is ∼1.6% in i.i.d. evaluation (2.3% at the default entropy setting). Under the persistent migration mechanic, catch rate is bounded by the fraction of sessions whose *counterfactual* server load exceeds the drop threshold in the step they are migrated—a noisy, low-probability event in a healthy cluster. Two directions could improve this. First, a *predictive horizon* longer than one step—pre-migrating sessions that are likely to drop within the next H steps—would widen the opportunity window. Second, a *model-based* rollout (learning a dynamics model of server load evolution and planning ahead) could exploit the degradation ramp more aggressively, as the Oracle baseline (≈68% catch rate in the crash scenario, Table 2) demonstrates the theoretical ceiling.

Reward sensitivity.. The reward function combines two terms: latency saving and bandwidth penalty, weighted by $C_{\text{bw}} = 5 \times 10^{-5}$ per KB. This weight was set manually. A systematic search over C_{bw} could discover configurations that reduce the 65% false-positive rate without sacrificing catch rate. The tradeoff is well-defined and likely Pareto-optimal, but the current paper reports only a single point.

Multi-agent coordination.. ProactiveHop uses a centralised controller that observes the full cluster state and issues per-server migration actions. In very large clusters ($N > 256$) or geo-distributed deployments, a centralised controller introduces a single point of failure and a communication bottleneck. A decentralised variant—where each server runs a local GNN policy using only its 2-hop neighbourhood—would preserve the topology-awareness advantage while eliminating the central dependency. The GraphSAGE architecture already computes per-node embeddings from local neighbourhoods, making this a natural architectural extension.

9. Conclusion

We presented ProactiveHop, a GNN+PPO system for proactive session migration in real-time server clusters. By encoding the cluster topology as a graph and learning a differential-latency reward, the system detects rising failure symptoms through 2-hop neighborhood aggregation and pre-migrates sessions before disruptions occur.

Empirically, PPO+GNN achieves the highest pre-migration rate (24.8%±7.5%) and lowest coefficient of variation (CoV = 0.30) among all learned policies in a controlled crash scenario (5 seeds), outperforming both PPO+MLP (17.7%, CoV = 0.42) and LSTM (17.6%, CoV = 0.63). In i.i.d. evaluation, GNN achieves 1.8× higher catch rate than MLP (1.64% vs 0.90%, p=0.0006) across 20 held-out seeds, and scales to 4× larger clusters with only 1.1× inference overhead vs 5.5× for LSTM.

The primary limitations and directions for future work are discussed in §8. Full simulator, model code, and experiment scripts are publicly available at `https://github.com/subodhwandile/proactivehop`.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT Author Statement

Subodh Wandile: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of Competing Interests

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The simulator, model code, training scripts, pre-trained checkpoints, and all experiment results that support the findings of this study are openly available at `https://github.com/subodhwandile/proactivehop`. The Alibaba 2018 cluster trace used in the trace-fidelity experiment is publicly available at `https://github.com/alibaba/clusterdata`.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

GitHub Copilot (code completion) was used during software development. Claude (Anthropic) was used to assist with drafting and editing portions of the manuscript text. All scientific content, experimental design, results, and conclusions are the sole responsibility of the author.

References

[1] Amazon Web Services, Use Spot instances with Amazon GameLift, `https://docs.aws.amazon.com/gamelift/latest/developerguide/fleets-spot.html`, accessed: 2025 (2024).

[2] Google / Agones Contributors, Health checking — Agones documentation, `https://agones.dev/site/docs/guides/health-checking/`, accessed: 2025 (2024).

[3] T. Levent-Levi, ICE restart — WebRTC glossary, `https://bloggeek.me/webrtcglossary/ice-restart/`, accessed: 2025 (2024).

[4] Mozilla Developer Network, RTCPeerConnection.restartIce() — MDN web docs, `https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection/restartIce`, accessed: 2025 (2024).

[5] B. Grozev, Support in-place ICE restart with a new Agent — Jitsi Videobridge issue #2438, `https://github.com/jitsi/jitsi-videobridge/issues/2438`, accessed: 2025 (2024).

[6] Riot Games Engineering, Fixing the Internet for real-time applications: Part I, https://www.riotgames.com/en/news/fixing-internet-real-time-applications-part-i, accessed: 2025 (2015).

[7] S. Abbasloo, C.-Y. Tsai, Y. Xu, H. J. Chao, Staying alive: Connection path migration for mobile realtime applications, in: 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), USENIX Association, 2021.

[8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, in: Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2019.

[9] H. Yan, K.-C. Chen, L.-H. Hsieh, Deep reinforcement learning-based joint handoff and resource management in heterogeneous networks, in: IEEE International Conference on Communications (ICC), IEEE, 2019.

[10] H. Mao, R. Netravali, M. Alizadeh, Neural adaptive video streaming with pensieve, in: Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2017.

[11] L. Chen, J. Lingys, K. Chen, F. Liu, AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization, in: Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), ACM, 2018.

[12] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, A. Warfield, Live migration of virtual machines, in: 2nd Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2005.

[13] A. v. d. Oord, Y. Li, O. Vinyals, Representation learning with contrastive predictive coding, in: arXiv preprint arXiv:1807.03748, 2018.

[14] N. Zhao, J. Chen, Z. Wang, X. Peng, G. Wang, Y. Wu, F. Zhou, N. Feng, W. Li, R. Ding, et al., Predicting node failure in cloud service systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2020.

[15] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, in: arXiv preprint arXiv:1707.06347, 2017.

[17] L. A. Barroso, J. Clidaras, U. Hölzle, The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines, 2nd Edition, Morgan & Claypool, 2013.

[18] Alibaba Group, Alibaba cluster trace program, `https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018`, 370K container events across 4,005 machines over 8 days. Accessed: 2025 (2018).

Appendix A. Hyperparameter Table

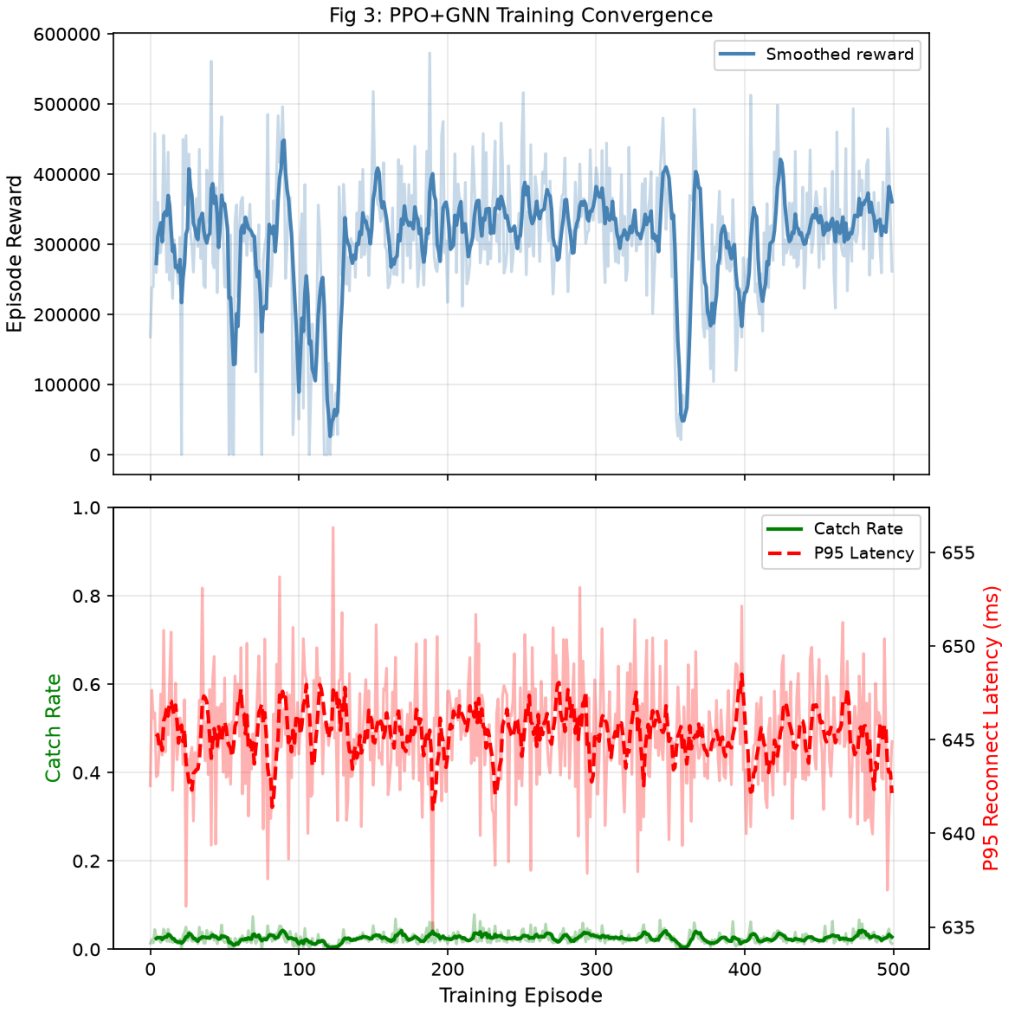


Figure 2: Training curve over 500 episodes. Reward grows from $\approx 10^5$ at episode 0 to $\sim 5 \times 10^5$ by episode 200, with continued improvement to episode 500. Best catch rate observed: 7.9%.

Table A.7: ProactiveHop hyperparameters (referenced from §4).

Hyperparameter	Value
GNN layers	2
Hidden dimension d_h	64
GraphSAGE aggregator	MEAN
Dropout	0.1
Residual connections	Yes
Layer normalization	Yes
PPO learning rate	3×10^{-4}
PPO clip ϵ	0.2
GAE λ	0.95
Entropy coefficient	0.05
Value loss coefficient	0.5
Max gradient norm	0.5
PPO update epochs	4
Training episodes	500
Steps per episode	100
Sessions per activation	20
$\bar{L}_{\text{react}}$	450 ms
$\bar{L}_{\text{spec}}$	5 ms
C_{bw}	5×10^{-5}
Base load ℓ_0	0.55
Spike probability	0.08
Spike magnitude	0.35